

Recurrent Neural Networks (RNNs)

209

209

Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a type of neural network that are designed to process sequences of data, such as time series or natural language text. They are called "recurrent" because they have a feedback loop, or recurrence, that allows information to persist in the network across time steps.

RNNs consist of multiple layers of neurons, each of which is connected to the previous layer in a time-dependent manner. This allows RNNs to maintain an internal state, which allows them to process sequences of data. The internal state is represented by a hidden state vector, which is updated at each time step based on the input and the previous hidden state.

One of the main building blocks of an RNN is the recurrent layer, which is composed of a set of recurrent neurons. Each recurrent neuron receives the input at the current time step and the hidden state at the previous time step, and it produces the output and the hidden state at the current time step. The output and the hidden state are used to make predictions or decisions based on the input sequence.

RNNs can be trained using supervised learning algorithms, such as backpropagation through time (BPTT), to learn the relationship between the input sequence and the output sequence. The training process involves adjusting the weights and biases of the recurrent neurons, so that the output of the network matches the desired output.

RNNs are commonly used in **natural language processing**, **speech recognition**, and **time-series prediction**. They are also used in video analysis, music generation, and speech synthesis.

RNNs can be extended to other architectures like LSTM and GRU which have been proven to tackle the problem of vanishing gradients

210

210

Recurrent Neural Networks (RNNs)

Feedforward networks map one input to one output

RNN inputs and outputs can vary in length, and different types of RNNs are used for different use cases, such as music generation, sentiment classification, and machine translation.

RNN works better than a simple neural network when data is sequential like Time-Series data and text data.

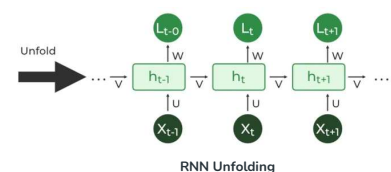
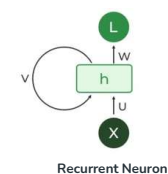
211

211

How RNN differs from Feedforward Neural Network?

The fundamental processing unit in a Recurrent Neural Network (RNN) is a **Recurrent Unit**, which is not explicitly called a “**Recurrent Neuron**.” This unit has the unique ability to maintain a hidden state, allowing the network to capture sequential dependencies by remembering previous inputs while processing.

[Long Short-Term Memory \(LSTM\)](#) and [Gated Recurrent Unit \(GRU\)](#) versions improve the RNN's ability to handle long-term dependencies.

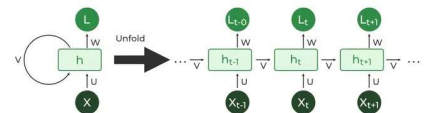


212

212

What is Recurrent Neural Network (RNN)?

- It is a type of Neural Network where the output from the previous step is fed as input to the current step.
- In traditional NN, all the inputs and outputs are independent of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words.
- RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its Hidden state, which remembers some information about a sequence.
- The state is also referred to as **Memory State** since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.



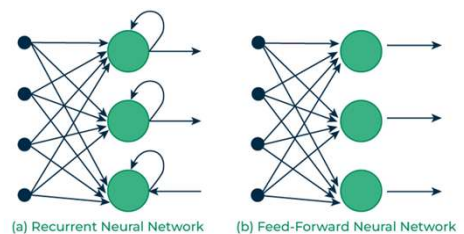
213

213

How RNN differs from Feedforward Neural Network?

ANN that do not have looping nodes are called feed forward neural networks. Because all information is only passed forward, this kind of NN is also referred to as a multi-layer neural network.

Information moves from the input layer to the output layer – if any hidden layers are present – unidirectionally in a feedforward neural network. These networks are appropriate for image classification tasks, for example, where input and output are independent. Nevertheless, their inability to retain previous inputs automatically renders them less useful for sequential data analysis



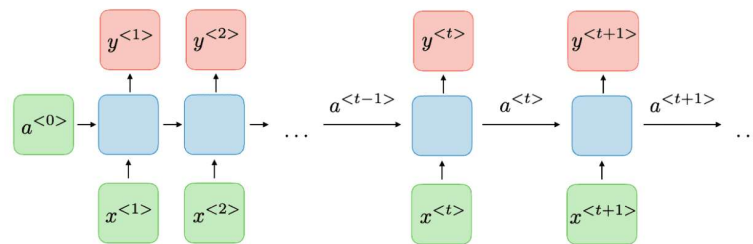
214

214

RNN: Basic architecture

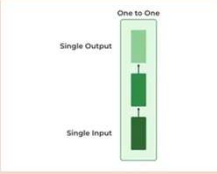
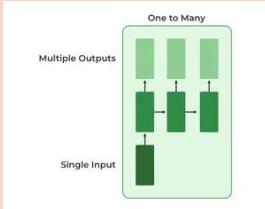
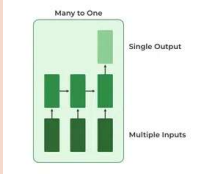
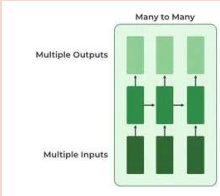
RNN consists of a series of repeating modules that are called cells and process the input data sequentially:

- the first cell takes as input the first sample of the sequence,
- the second cell takes the second sample, and so on...



215

Types Of RNN

One to One	One to Many	Many to One	Many to Many
this type of RNN behaves the same as any simple Neural network it is also known as Vanilla Neural Network. In this Neural network, there is only one input and one output. 	there is one input and many outputs associated with it. One of the most used examples of this network is Image captioning where given an image we predict a sentence having Multiple words. 	this type of RNN behaves the same as any simple Neural network it is also known as Vanilla Neural Network. In this Neural network, there is only one input and one output. 	there is one input and many outputs associated with it. One of the most used examples of this network is Image captioning where given an image we predict a sentence having Multiple words. 

<https://www.geeksforgeeks.org/introduction-to-recurrent-neural-network/>

216

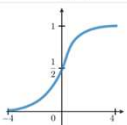
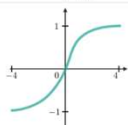
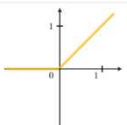
216

Handling long term dependencies

• **Vanishing/exploding gradient** The vanishing and exploding gradient phenomena are often encountered in the context of RNNs. The reason why they happen is that it is difficult to capture long term dependencies because of multiplicative gradient that can be exponentially decreasing/increasing with respect to the number of layers.

• **Gradient clipping** - It is a technique used to cope with the exploding gradient problem sometimes encountered when performing backpropagation. By capping the maximum value for the gradient, this phenomenon is controlled in practice.

Commonly used activation functions:

Sigmoid	Tanh	RELU
$g(z) = \frac{1}{1 + e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$
		

217

217

Common Problems of RNN's

Exploding gradients: This is when the algorithm, without much reason, assigns a stupidly high importance to the weights. Fortunately, this problem can be easily solved by truncating or squashing the gradients.

Vanishing gradients: These occur when the values of a gradient are too small and the model stops learning or takes way too long as a result. Fortunately, it was solved through the concept of LSTM by Sepp Hochreiter and Juergen Schmidhuber.

Complex training process: Because RNNs process data sequentially, this can result in a tedious training process.

Difficulty with long sequences: The longer the sequence, the harder RNNs must work to remember past information.

Inefficient methods: RNNs process data sequentially, which can be a slow and inefficient approach.

218

218

Recurrent Neural Network

Advantages	Drawbacks
Possibility of processing input of any length	Computation being slow
Model size not increasing with size of input	Difficulty of accessing information from a long time ago
Computation takes into account historical information	Cannot consider any future input for the current state
• Weights are shared across time	

219

219

Variant RNN architectures

Bidirectional recurrent neural networks (BRNN): These are a variant network architecture of RNNs. While unidirectional RNNs can only draw from previous inputs to make predictions about the current state, bidirectional RNNs pull in future data to improve the accuracy of it.

Long short-term memory (LSTM): This is a popular RNN architecture, was introduced by Sepp Hochreiter and Juergen Schmidhuber as a solution to the vanishing gradient problem, and address the problem of long-term dependencies. That is, if the previous state that is influencing the current prediction is not in the recent past, the RNN model may not be able to accurately predict the current state.

Gated recurrent units (GRUs): This RNN variant is similar the LSTMs as it also works to address the short-term memory problem of RNN models.

- instead of using a “cell state” regulate information, it uses hidden states, and instead of three gates, it has two—a reset gate and an update gate.
- similar to the gates within LSTMs, the reset and update gates control how much and which information to retain.

deep RNN: are more efficient than their shallow models. In “shallow” RNN, there are generally three blocks for computation of parameters: the input state, the hidden state, and the output state. These blocks are associated with a single weight matrix corresponding to a shallow transformation which can be represented by a single-layer multilayer perceptron (MLP). In deep RNN, the state of the RNN can be decomposed into multiple layers.

220

220

RNN applications

Feedforward networks map one input to one output,

RNN inputs and outputs can vary in length, and different types of RNNs are used for different use cases, such as music generation, sentiment classification, and machine translation.

RNN works better than a simple neural network when data is sequential like Time-Series data and text data.

221

221

TP7 – RNNs Introduction & Architecture

APRENDIZAGEM PROFUNDA – APPROF

MESTRADO EM ENGENHARIA INFORMÁTICA – ENGENHARIA DE DADOS

222

Index

- **Types of Architectures**
 - Simple RNN
 - LTM
 - Gru
 - Deep
 - Bidirectional

223

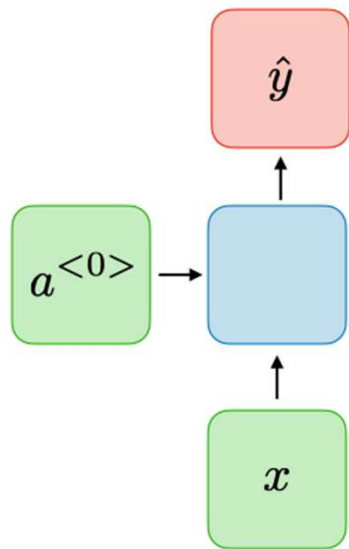
Types of Architectures

Recurrent Neural Networks (RNNs) come in various architectures, each designed to address specific challenges or performance for particular tasks.

Some common types of RNN architectures:

- Simple RNN
- **Long Short-Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**
- Bidirectional RNN
- Deep RNN

224



Simple RNN

Simple RNN: This is the basic form of RNN, where each neuron in the hidden layer is connected to every other neuron, including itself, through recurrent connections.

While simple to understand and implement, simple RNNs can suffer from the **vanishing gradient** problem, **limiting their ability to capture long-term dependencies**.

225

Advantages

- **Simplicity:** Simple RNNs are straightforward to understand and implement, making them easy to use for basic sequential data tasks.
- **Real-Time Processing:** They can process input sequences in real-time, making them suitable for tasks that require immediate feedback or prediction.

Disadvantages

- **Difficulty in Capturing Long-Term Dependencies:** Simple RNNs struggle to capture long-term dependencies in sequential data, leading to limitations in modeling complex relationships.
- **Vanishing Gradient:** Training Simple RNNs can suffer from the vanishing gradient problem, hindering learning and convergence, particularly in deep architectures or with long sequences.
- **Limited Memory Capacity:** They have a limited memory capacity, which can restrict their ability to retain information over long sequences.

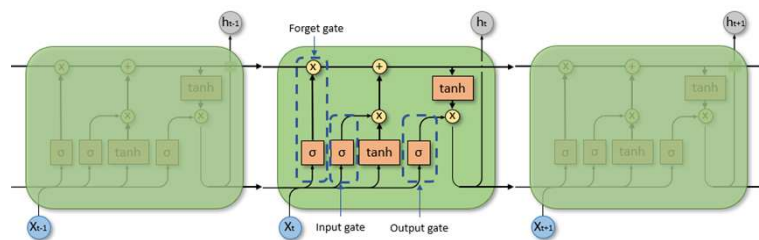
226

Long Short-Term Memory (LSTM):

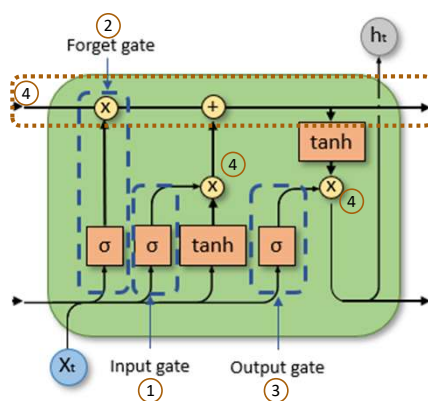
Long Short-Term Memory (LSTM): LSTMs are a type of RNN architecture introduced to address the vanishing gradient problem.

They incorporate specialized **memory** cells with gating mechanisms that regulate the flow of information, allowing LSTMs to capture long-term dependencies more effectively.

The key components of an LSTM cell include **input**, **forget**, and **output gates**, as well as a **cell state**.



227



LSTM componentes

Key components of an LSTM cell include:

- 1. Input Gate:** This gate regulates the flow of information into the cell state. It determines which information from the current input and the previous hidden state should be stored in the cell state.
- 2. Forget Gate:** The forget gate controls the retention or deletion of information from the cell state. It decides which information is no longer relevant and should be forgotten or discarded from the cell state.
- 3. Output Gate:** The output gate determines the information that will be output from the cell state to the next time step's hidden state. It selects relevant information from the cell state to be passed on as the output of the LSTM cell.
- 4. Cell State:** The cell state serves as the internal memory of the LSTM cell. It carries information across time steps and is modified by the input gate, forget gate, and output gate while also being influenced by the current input and previous hidden state.

228

Advantages LSTM

- **Capturing Long-Term Dependencies:** LSTMs allow for modelling sequences with long-range dependencies.
- **Memory Retention:** They maintain a more sophisticated memory, allowing relevant information to persist over multiple time steps.
- **Gating Mechanisms:** LSTMs utilize gates to selectively update and utilize information, enhancing their ability to focus on important features.
- **Robustness to Noisy Data:** They are inherently robust to noisy or missing data, making them suitable for real-world applications.
- **Versatility:** LSTMs can be applied to a wide range of sequential data processing tasks, demonstrating flexibility and effectiveness across diverse domains.

229

Disadvantages LSTM

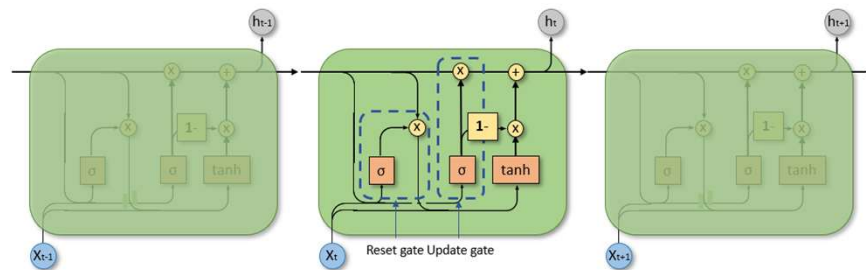
- **Complexity:** LSTMs are more complex than traditional RNNs, making them harder to understand and implement. The intricate architecture of LSTMs can make it challenging to interpret the learned representations.
- **Training Time:** Training LSTMs can be computationally intensive and time-consuming, especially for large-scale datasets and deep architectures.
- **Overfitting:** They are susceptible to overfitting, particularly when trained on small datasets or with high model capacity.
- **Hyperparameter Tuning:** LSTMs require careful tuning of hyperparameters for optimal performance, which can be time-consuming.

230

Gated Recurrent Unit (GRU):

Gated Recurrent Unit (GRU): GRUs are another variant of RNNs that address the **vanishing gradient** problem while **simplifying** the architecture compared to LSTMs.

GRUs combine the input and forget gates into a single update gate and merge the cell state and hidden state, resulting in **fewer parameters and faster training compared to LSTMs**.



231

Advantages GRU



Simplicity

GRUs are simpler than LSTMs, with fewer parameters and computations, making them easier to train and implement.



Faster Training

They typically converge faster during training compared to LSTMs due to their simpler architecture.



Memory Efficiency

GRUs require less memory compared to LSTMs, making them more memory-efficient, particularly for large-scale models.



Performance

In some cases, GRUs have been shown to achieve comparable performance to LSTMs on various tasks while being computationally more efficient.

232

Disadvantages

GRU



Limited Memory

GRUs may have limited capacity to capture long-term dependencies compared to LSTMs.



Less Control

They have fewer parameters for controlling information flow compared to LSTMs, potentially leading to less precise control over memory management.



Performance Variability

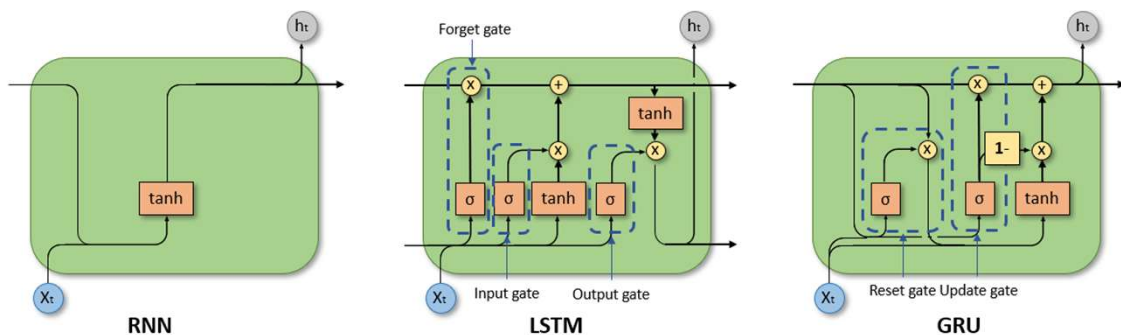
While GRUs can perform well in many cases, they may not always outperform LSTMs, and their performance can vary depending on the specific task and dataset.



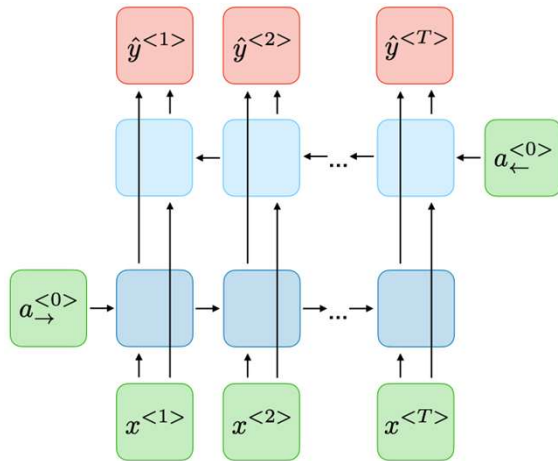
Difficulty in Interpretability

Like LSTMs, GRUs can be challenging to interpret due to their complex internal mechanisms.

233



234



Bidirectional RNN

Bidirectional RNN: In a bidirectional RNN, information flows in both forward and backward directions through time.

This architecture allows the network to consider past and future context when making predictions, which can be beneficial for tasks where contextual information from both directions is important, such as speech recognition and sentiment analysis.

235

Advantages Bidirectional RNN



Contextual Understanding

Bidirectional RNNs capture both past and future context, providing a more comprehensive understanding of the input sequence.



Improved Performance

They can yield better performance compared to unidirectional RNNs by leveraging information from both directions.



Robustness

Bidirectional RNNs are more robust to noise and ambiguity in the input sequence, as they consider multiple contexts.

236

Disadvantages

Bidirectional RNN



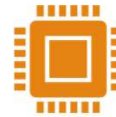
Increased Complexity

Bidirectional RNNs are more complex and computationally intensive than unidirectional RNNs due to processing sequences in both directions.



Delayed Prediction

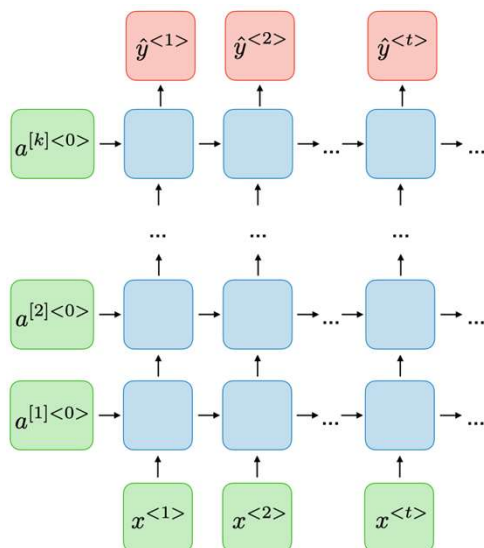
They require knowledge of future inputs, making them unsuitable for real-time applications where all input data may not be available in advance.



Memory Consumption

Bidirectional RNNs may consume more memory due to the need to store activations from both directions during training and inference.

237



Deep RNN

Deep RNN: Deep RNNs consist of multiple layers of recurrent units stacked on top of each other.

Deep architectures can capture more complex hierarchical patterns in the data but may be more prone to vanishing or exploding gradients during training.

Techniques such as gradient clipping and careful initialization of parameters are often used to mitigate these issues.

238

Advantages Deep RNN

- **Hierarchical Representation:** Deep RNNs can capture hierarchical features and abstract representations of sequential data, potentially leading to improved performance.
- **Enhanced Learning Capacity:** They have a larger model capacity, allowing them to learn more complex patterns and dependencies in the data.
- **Better Generalization:** Deep RNNs can generalize well to unseen data when trained on large and diverse datasets.



239

Disadvantages Deep RNN

- **Increased Complexity:** Deep RNNs are more complex and computationally intensive, requiring more parameters and longer training times.
- **Vanishing/Exploding Gradients:** Training deep RNNs can be challenging due to the vanishing or exploding gradient problem, which can hinder learning and convergence.
- **Overfitting:** Deep RNNs are prone to overfitting, particularly when trained on small datasets or with insufficient regularization techniques. Regularization and careful tuning are necessary to mitigate this issue.

240